

```

import React, {
  forwardRef,
  useState,
  useEffect,
  useCallback,
  useImperativeHandle,
  useRef,
} from "react";
import { ZoomIn, ZoomOut, RotateCw, RefreshCw } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface ImageViewerProps extends React.HTMLAttributes<HTMLDivElement> {
  src?: string;
  alt?: string;
  onScaleChange?: (scale: number, rotate: number) => void;
}

export interface ImageViewerRef {
  reset: () => void;
  zoomIn: () => void;
  zoomOut: () => void;
  rotate: () => void;
}

const MIN_SCALE = 0.1;
const MAX_SCALE = 5;
const SCALE_STEP = 0.2;

export const ImageViewer = forwardRef<ImageViewerRef, ImageViewerProps>(
  (
    {
      src,
      alt = "Document preview",
      onScaleChange,
      className,
      style,
      ...props
    },
    ref,
  ) => {
    const [scale, setScale] = useState(1);
    const [rotate, setRotate] = useState(0);
    const [isLoading, setIsLoading] = useState(false);
    const [hasError, setHasError] = useState(false);
    const imgRef = useRef<HTMLImageElement>(null);
  }

```

```

const containerRef = useRef<HTMLDivElement>(null);

const clampScale = useCallback((newScale: number) => {
  return Math.max(MIN_SCALE, Math.min(MAX_SCALE, newScale));
}, []);

const handleScaleChange = useCallback(
  (newScale: number) => {
    const clamped = clampScale(newScale);
    setScale(clamped);
    onScaleChange?.(clamped, rotate);
  },
  [clampScale, onScaleChange, rotate],
);

const handleRotateChange = useCallback(
  (newRotate: number) => {
    const normalized = ((newRotate % 360) + 360) % 360;
    setRotate(normalized);
    onScaleChange?.(scale, normalized);
  },
  [onScaleChange, scale],
);

const reset = useCallback(() => {
  setScale(1);
  setRotate(0);
  onScaleChange?.(1, 0);
}, [onScaleChange]);

const zoomIn = useCallback(() => {
  handleScaleChange(scale + SCALE_STEP);
}, [handleScaleChange, scale]);

const zoomOut = useCallback(() => {
  handleScaleChange(scale - SCALE_STEP);
}, [handleScaleChange, scale]);

const rotateCw = useCallback(() => {
  handleRotateChange(rotate + 90);
}, [handleRotateChange, rotate]);

useImperativeHandle(
  ref,
  () => ({
    reset,
    zoomIn,
    zoomOut,
    rotate: rotateCw,
  })),

```

```

    [reset, zoomIn, zoomOut, rotateCw],
  );

  useEffect(() => {
    const handleKeyDown = (event: KeyboardEvent) => {
      if (event.key === "Escape") {
        reset();
      }
    };

    document.addEventListener("keydown", handleKeyDown);
    return () => document.removeEventListener("keydown", handleKeyDown);
  }, [reset]);

  const handleImageLoad = useCallback(() => {
    setIsLoading(false);
    setHasError(false);
  }, []);

  const handleImageError = useCallback(() => {
    setIsLoading(false);
    setHasError(true);
  }, []);

  const handleDoubleClick = useCallback(() => {
    reset();
  }, [reset]);

  const handleWheel = useCallback(
    (event: React.WheelEvent<HTMLDivElement>) => {
      if (event.ctrlKey || event.metaKey) {
        event.preventDefault();
        const delta = event.deltaY > 0 ? -SCALE_STEP : SCALE_STEP;
        handleScaleChange(scale + delta);
      }
    },
    [handleScaleChange, scale],
  );

  if (!src) {
    return (
      <div
        ref={containerRef}
        className={cn(
          "relative flex items-center justify-center",
          "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
          "border",
          "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
          "rounded-surface",
          "min-h-[300px]",

```

```

        className,
      )}
      style={style}
      {...props}
    >
      <div className="text-center p-8">
        <RefreshCw
          className="w-12 h-12 text-muted-foreground/40 mx-auto mb-3
animate-spin"
          aria-hidden="true"
        />
        <p className="text-muted-foreground">No image source provided</p>
      </div>
    </div>
  );
}

return (
  <div
    ref={containerRef}
    className={cn(
      "relative",
      "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
      "border border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
      "rounded-surface",
      "overflow-hidden",
      className,
    )}
    style={style}
    onWheel={handleWheel}
    {...props}
  >
    <div
      className={cn(
        "flex items-center justify-between p-3",
        "bg-[color-mix(in_oklch,var(--color-card)_95%,transparent)]",
        "backdrop-blur-[var(--backdrop-blur)]",
        "border-b
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
      )}
      role="toolbar"
      aria-label="Image viewer controls"
    >
      <div className="flex items-center gap-2">
        <span className="text-xs text-muted-foreground font-mono">
          {Math.round(scale * 100)}%
        </span>
        {rotate !== 0 && (
          <span className="text-xs text-muted-foreground font-mono">
            {rotate}°
          </span>
        )}
      </div>
    </div>
  >

```

```

        </span>
    })
</div>
<div className="flex items-center gap-1.5">
    <button
        type="button"
        onClick={zoomOut}
        disabled={scale <= MIN_SCALE}
        className={cn(
            "relative flex items-center justify-center",
            "w-8 h-8 rounded-md",
            "text-muted-foreground/60 hover:text-foreground",
            "bg-transparent hover:bg-accent",
            "transition-transform duration-100",
            "active:scale-95",
            "focus-visible:outline-none",
            "focus-visible:ring-2 focus-visible:ring-amber-500/20
focus-visible:border-amber-500",
            "disabled:opacity-50 disabled:pointer-events-none",
        )}
        aria-label="Zoom out"
        aria-disabled={scale <= MIN_SCALE}
    >
        <ZoomOut className="w-4 h-4" aria-hidden="true" />
    </button>
    <button
        type="button"
        onClick={reset}
        className={cn(
            "relative flex items-center justify-center",
            "w-8 h-8 rounded-md",
            "text-muted-foreground/60 hover:text-foreground",
            "bg-transparent hover:bg-accent",
            "transition-transform duration-100",
            "active:scale-95",
            "focus-visible:outline-none",
            "focus-visible:ring-2 focus-visible:ring-amber-500/20
focus-visible:border-amber-500",
        )}
        aria-label="Reset view"
    >
        <RefreshCw className="w-4 h-4" aria-hidden="true" />
    </button>
    <button
        type="button"
        onClick={zoomIn}
        disabled={scale >= MAX_SCALE}
        className={cn(
            "relative flex items-center justify-center",
            "w-8 h-8 rounded-md",

```

```

        "text-muted-foreground/60 hover:text-foreground",
        "bg-transparent hover:bg-accent",
        "transition-transform duration-100",
        "active:scale-95",
        "focus-visible:outline-none",
        "focus-visible:ring-2 focus-visible:ring-amber-500/20
focus-visible:border-amber-500",
        "disabled:opacity-50 disabled:pointer-events-none",
    ))
    aria-label="Zoom in"
    aria-disabled={scale >= MAX_SCALE}
  >
    <ZoomIn className="w-4 h-4" aria-hidden="true" />
  </button>
  <button
    type="button"
    onClick={rotateCw}
    className={cn(
      "relative flex items-center justify-center",
      "w-8 h-8 rounded-md",
      "text-muted-foreground/60 hover:text-foreground",
      "bg-transparent hover:bg-accent",
      "transition-transform duration-100",
      "active:scale-95",
      "focus-visible:outline-none",
      "focus-visible:ring-2 focus-visible:ring-amber-500/20
focus-visible:border-amber-500",
    ))
    aria-label="Rotate 90° clockwise"
  >
    <RotateCw className="w-4 h-4" aria-hidden="true" />
  </button>
</div>
</div>
<div
  className="relative w-full h-full min-h-[300px] overflow-auto"
  style={{
    backgroundColor:
      "color-mix(in oklch, var(--color-muted) 30%, transparent)",
  }}
>
  {isLoading && (
    <div className="absolute inset-0 flex items-center justify-center z-10">
      <RefreshCw
        className="w-10 h-10 text-muted-foreground/40 animate-spin"
        aria-hidden="true"
      />
      <span className="sr-only">Loading image...</span>
    </div>
  )}

```

```

      {hasError && (
        <div className="absolute inset-0 flex items-center justify-center z-10
p-8 text-center">
          <div>
            <RefreshCw
              className="w-12 h-12 text-destructive/60 mx-auto mb-3"
              aria-hidden="true"
            />
            <p className="text-destructive font-medium">
              Failed to load image
            </p>
            <p className="text-sm text-muted-foreground mt-1">
              The image source may be invalid or inaccessible
            </p>
          </div>
        </div>
      )}
    <img
      ref={imgRef}
      src={src}
      alt={alt}
      className={cn(
        "block max-w-none",
        "transition-transform duration-200 ease-out",
        "origin-center",
        isLoading && "opacity-0",
        hasError && "hidden",
      )}
      style={{
        transform: `scale(${scale}) rotate(${rotate}deg)`,
        transformOrigin: "center center",
      }}
      onLoad={handleImageLoad}
      onError={handleImageError}
      onDoubleClick={handleDoubleClick}
      onLoadStart={() => setIsLoading(true)}
    />
  </div>
</div>
);
},
);

```

```

ImageViewer.displayName = "ImageViewer";

```